

MoveMate



Alan Bissio

E.E.S.T. N°1 Vicente López

7mo 2da

Proyecto de Implementación de Sitios Web Dinámicos

Jorge Guzman

22 de abril de 2026

Índice

1. Introducción.....	3
2. Objetivo de la actividad.....	5
3. Tecnologías utilizadas.....	6
3.1. React Native.....	6
3.2. Expo.....	6
3.3. TypeScript.....	7
3.4. Expo Router.....	7
3.5. React Native Reanimated.....	7
3.6. React Native Gesture Handler.....	8
3.7. Expo Location.....	8
3.8. WebView, Leaflet y OpenStreetMap.....	8
3.9. AsyncStorage.....	10
3.10. Firebase.....	11
Firebase Authentication.....	11
Cloud Firestore.....	11
3.11. pnpm.....	11
3.12. EAS Build.....	12
4. Arquitectura de la solución.....	13
5. Pantalla principal.....	16
6. Inicio de actividades.....	18
7. Seguimiento de ubicación y mapa.....	19
8. Animaciones.....	20
9. Historial de actividades.....	21
10. Gestos.....	23
11. Almacenamiento local.....	25
12. Firebase y sincronización en la nube.....	26
13. Tema claro y oscuro.....	27
14. Generación de APK.....	29
15. Pruebas de funcionamiento.....	30
16. Dificultades encontradas.....	31
17. Resultado final.....	33
18. Conclusión.....	34

1. Introducción

En esta actividad se desarrolló **MoveMate**, una aplicación móvil orientada al seguimiento de actividad física diaria.

La aplicación fue desarrollada utilizando **React Native y Expo**, permitiendo crear una interfaz preparada para dispositivos móviles Android y trabajar con funciones propias del teléfono, como la ubicación mediante GPS.

MoveMate permite iniciar diferentes tipos de actividades físicas, entre ellas caminata, carrera y bicicleta. Durante una actividad se registra el tiempo transcurrido, la ubicación del usuario, la distancia recorrida y una estimación de las calorías consumidas.

Además, la aplicación incorpora un historial donde se almacenan las actividades realizadas, filtros por tipo y fecha, eliminación mediante gestos, sistema de tema claro y oscuro y sincronización de información mediante Firebase.

Para representar los recorridos se utilizó un mapa basado en **Leaflet y OpenStreetMap**, mostrado dentro de la aplicación mediante WebView.

Finalmente, se utilizó **EAS Build** para preparar una versión instalable de la aplicación para Android.



2. Objetivo de la actividad

El objetivo principal de la actividad fue desarrollar una aplicación móvil utilizando **React Native y Expo** que permitiera registrar y consultar actividades físicas.

La implementación debía incorporar diferentes conceptos propios del desarrollo móvil:

- Crear una pantalla principal con un resumen diario.
- Permitir seleccionar distintos tipos de actividad.
- Obtener la ubicación del dispositivo mediante GPS.
- Mostrar el recorrido realizado sobre un mapa.
- Utilizar animaciones mediante React Native Reanimated.
- Mantener un historial de actividades.
- Incorporar filtros por tipo de actividad y fecha.
- Utilizar gestos dentro de la interfaz.
- Guardar información localmente.
- Sincronizar actividades mediante un servicio en la nube.
- Incorporar un tema claro y oscuro.
- Generar una versión instalable para Android.

De esta manera, el proyecto permitió trabajar con interfaces móviles, sensores del dispositivo, almacenamiento local, servicios externos, animaciones y compilación de aplicaciones.

3. Tecnologías utilizadas

Para desarrollar MoveMate se utilizaron diferentes tecnologías y herramientas.

3.1. React Native

React Native fue utilizado como tecnología principal para desarrollar la interfaz de la aplicación.

Permite crear aplicaciones móviles utilizando componentes y conceptos similares a React.

Dentro del proyecto se utilizó para desarrollar:

- Pantallas.
- Botones.
- Formularios.
- Tarjetas.
- Historial.
- Controles de actividad.
- Temas.
- Interfaz adaptable a dispositivos móviles.

3.2. Expo

Expo fue utilizado como entorno principal para desarrollar y probar MoveMate.

La versión del SDK utilizada durante el proyecto fue:

Expo SDK 57

Expo facilitó principalmente:

- Ejecución de la aplicación en dispositivos Android.
- Pruebas mediante Expo Go.
- Acceso a ubicación.
- Configuración del proyecto.
- Integración con diferentes librerías.
- Generación de versiones Android mediante EAS Build.

Durante el desarrollo se inició el proyecto mediante:

```
pnpm exec expo start
```

3.3. TypeScript

La aplicación fue desarrollada utilizando TypeScript.

Esto permitió definir tipos para diferentes elementos del sistema, principalmente las actividades y los puntos obtenidos mediante GPS.

Por ejemplo, se utilizaron tipos para almacenar:

- Tipo de actividad.
- Latitud.
- Longitud.
- Fecha y hora.
- Duración.
- Distancia.
- Calorías.
- Ruta registrada.

Esto ayudó a mantener una estructura más ordenada y reducir errores durante el desarrollo.

3.4. Expo Router

Expo Router fue utilizado para administrar la navegación entre las diferentes pantallas de MoveMate.

Las principales secciones son:

- Inicio.
- Historial.
- Perfil.
- Actividad en progreso.

La navegación principal utiliza una barra inferior con las opciones:

- Inicio.
- Historial.
- Perfil.

También se utiliza una ruta independiente cuando comienza una actividad.

3.5. React Native Reanimated

React Native Reanimated fue utilizado para incorporar animaciones dentro de la aplicación.

Entre las animaciones desarrolladas se encuentran:

- Entrada de las estadísticas del dashboard.
- Aparición de elementos principales.
- Animación del botón flotante para iniciar una actividad.
- Transiciones de distintos componentes.

Las animaciones permiten que la interfaz tenga movimientos más fluidos sin afectar considerablemente el rendimiento.

3.6. React Native Gesture Handler

React Native Gesture Handler fue utilizado para incorporar gestos dentro de la aplicación.

La principal implementación se encuentra en el historial.

Cada actividad puede eliminarse deslizando su tarjeta hacia la izquierda.

Al realizar el gesto aparece una sección roja con un icono y la opción de eliminar.

Esta implementación cumple con el requisito de incorporar un gesto complejo dentro de la aplicación.

3.7. Expo Location

Expo Location fue utilizado para obtener la ubicación del dispositivo.

Cuando se inicia una actividad, la aplicación solicita permiso para acceder a la ubicación.

Una vez autorizado, se obtiene información como:

- Latitud.
- Longitud.
- Momento en que se obtuvo la posición.

Mientras la actividad permanece activa, se siguen obteniendo nuevas posiciones.

Estos puntos permiten construir la ruta realizada por el usuario.

3.8. WebView, Leaflet y OpenStreetMap

Para mostrar el recorrido se utilizó una combinación de:

- React Native WebView.

- Leaflet.
- OpenStreetMap.

WebView permite mostrar contenido web dentro de una pantalla de React Native.

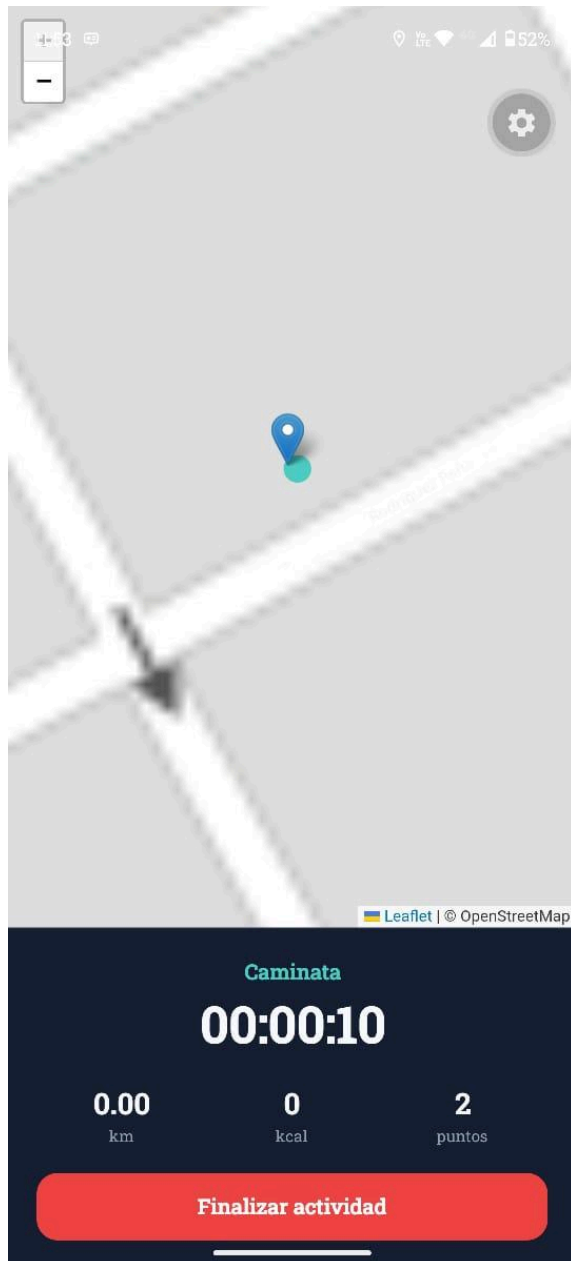
Dentro de esta vista se carga Leaflet, que se encarga de administrar el mapa.

Los mapas base son proporcionados por OpenStreetMap.

Sobre el mapa se muestra:

- Posición del usuario.
- Punto de inicio.
- Recorrido realizado.
- Calles y referencias geográficas.

Esta solución permitió mostrar mapas sin depender directamente de Google Maps Platform.



3.9. AsyncStorage

AsyncStorage fue utilizado como sistema de almacenamiento local.

Permite conservar información dentro del dispositivo incluso después de cerrar la aplicación.

Se utiliza principalmente para:

- Guardar actividades.
- Recuperar el historial.
- Mantener la preferencia de tema.

Las actividades se almacenan de manera separada según el usuario autenticado.

Cuando no existe una sesión iniciada, se utiliza un almacenamiento independiente para las actividades realizadas como invitado.

3.10. Firebase

Firebase fue utilizado para incorporar sincronización mediante servicios en la nube.

Se utilizaron principalmente dos servicios:

Firebase Authentication

Permite:

- Crear cuentas.
- Iniciar sesión.
- Cerrar sesión.
- Identificar a cada usuario mediante un ID único.

Cloud Firestore

Se utiliza para almacenar las actividades realizadas por cada usuario.

Cada actividad guarda información como:

- ID.
- Tipo de actividad.
- Inicio.
- Finalización.
- Duración.
- Distancia.
- Calorías.
- Ruta.
- ID del usuario.

Cuando el usuario inicia sesión, las actividades almacenadas en Firestore pueden recuperarse y mostrarse nuevamente en el historial.

3.11. pnpm

pnpm fue utilizado como gestor de paquetes del proyecto.

Entre los comandos utilizados se encuentran:

```
pnpm install
```

```
pnpm exec expo start
```

```
pnpm exec expo install
```

```
pnpm dlx eas-cli
```

Las dependencias del proyecto son administradas principalmente mediante:

```
package.json
```

3.12. EAS Build

EAS Build fue utilizado para generar una versión Android de MoveMate.

El proyecto fue vinculado con Expo y se configuró un perfil denominado:

```
preview
```

para generar archivos APK.

El comando utilizado fue:

```
pnpm dlx eas-cli build -p android --profile preview
```

La compilación se realiza en los servidores de Expo.

4. Arquitectura de la solución

MoveMate mantiene diferentes responsabilidades separadas mediante carpetas.

La estructura principal puede representarse de manera simplificada así:

MoveMate/

```

|
|— app/
|   |— (tabs)/
|   |   |— index.tsx
|   |   |— history.tsx
|   |   |— profile.tsx
|   |   |— _layout.tsx
|   |
|   |— activity/
|   |   |— active.tsx
|   |
|   |— _layout.tsx
|
|— components/
|   |— ActivityHistoryCard.tsx
|   |— ActivityMap.tsx
|   |— ActivityTypeModal.tsx
|   |— AnimatedStat.tsx
|   |— FloatingStartButton.tsx
|
|— constants/
```

```
|   └─ theme.ts
|
|
├─ hooks/
|   └─ useAppTheme.tsx
|   └─ useAuth.tsx
|
|
├─ services/
|   └─ auth.ts
|   └─ cloudActivities.ts
|   └─ firebase.ts
|   └─ location.ts
|   └─ storage.ts
|
|
├─ types/
|   └─ activity.ts
|
|
├─ app.json
├─ eas.json
├─ package.json
└─ pnpm-lock.yaml
```

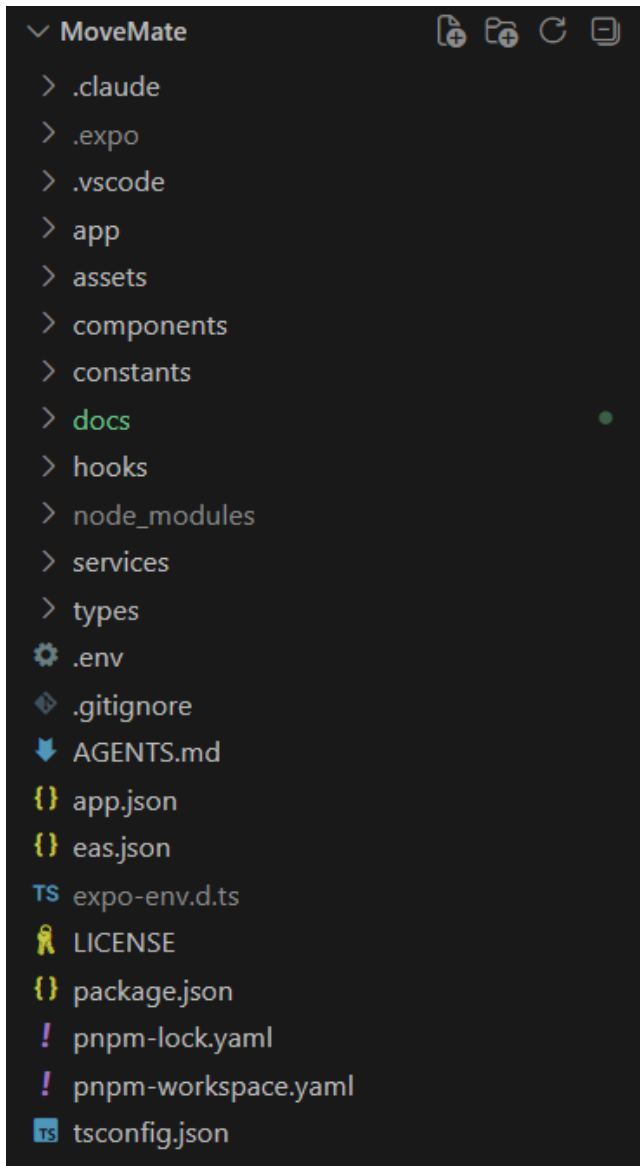
La carpeta **app** contiene principalmente las pantallas.

components contiene componentes reutilizables.

services agrupa funciones relacionadas con servicios externos y almacenamiento.

hooks contiene estados globales como autenticación y tema.

types contiene las estructuras utilizadas para representar las actividades.



5. Pantalla principal

La pantalla principal funciona como dashboard de MoveMate.

Muestra un resumen del movimiento diario mediante diferentes estadísticas.

Entre los datos mostrados se encuentran:

- Pasos.
- Distancia.
- Calorías.
- Porcentaje del objetivo diario.

También contiene un botón flotante que permite iniciar una nueva actividad.

Al presionar este botón aparece un selector con:

- Caminata.
- Carrera.
- Bicicleta.

La pantalla incorpora animaciones de entrada para mejorar la presentación de los datos.



6. Inicio de actividades

Cuando el usuario presiona el botón de nueva actividad se muestra una ventana donde puede seleccionar el tipo.

Las opciones disponibles son:

- Caminata.
- Carrera.
- Bicicleta.

Después de seleccionar una opción, la aplicación redirige a la pantalla de actividad activa.

Durante esta etapa comienza el seguimiento del tiempo y de la ubicación.

7. Seguimiento de ubicación y mapa

Al comenzar una actividad, MoveMate solicita permiso para utilizar la ubicación del dispositivo.

Después se obtiene la posición actual mediante Expo Location.

Las posiciones recibidas se almacenan como diferentes puntos de una ruta.

Cada punto contiene:

- Latitud.
- Longitud.
- Timestamp.

A medida que cambia la ubicación, la aplicación agrega nuevos puntos.

La distancia total se calcula utilizando las coordenadas obtenidas.

Además, se muestra:

- Tiempo transcurrido.
- Distancia.
- Calorías estimadas.
- Cantidad de puntos registrados.

El recorrido se representa visualmente mediante Leaflet y OpenStreetMap.

8. Animaciones

MoveMate incorpora diferentes animaciones mediante React Native Reanimated.

Se utilizaron principalmente para mejorar la pantalla principal.

Entre ellas se encuentran:

- Aparición progresiva de estadísticas.
- Desplazamiento de tarjetas.
- Animación del botón flotante.
- Transiciones de elementos.

Esto permite que la interfaz resulte más dinámica sin agregar controles innecesarios.

9. Historial de actividades

Cada actividad finalizada puede guardarse en el historial.

La información mostrada incluye:

- Tipo de actividad.
- Fecha.
- Distancia.
- Tiempo.
- Calorías.

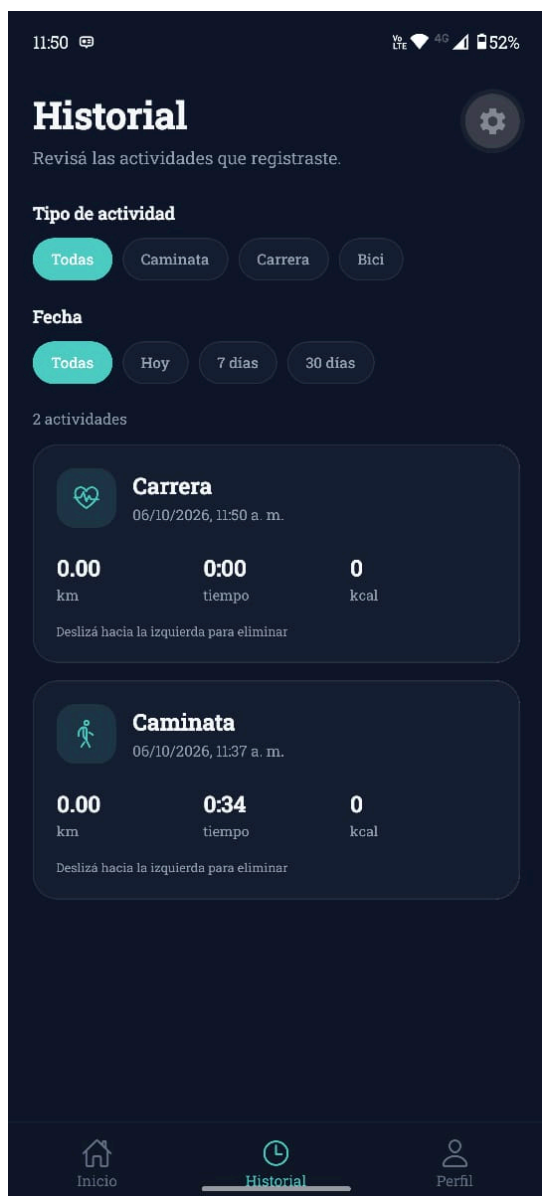
También se incorporaron filtros.

Por tipo de actividad:

- Todas.
- Caminata.
- Carrera.
- Bicicleta.

Por fecha:

- Todas.
- Hoy.
- Últimos 7 días.
- Últimos 30 días.



10. Gestos

Para incorporar gestos se desarrolló un sistema de eliminación mediante swipe.

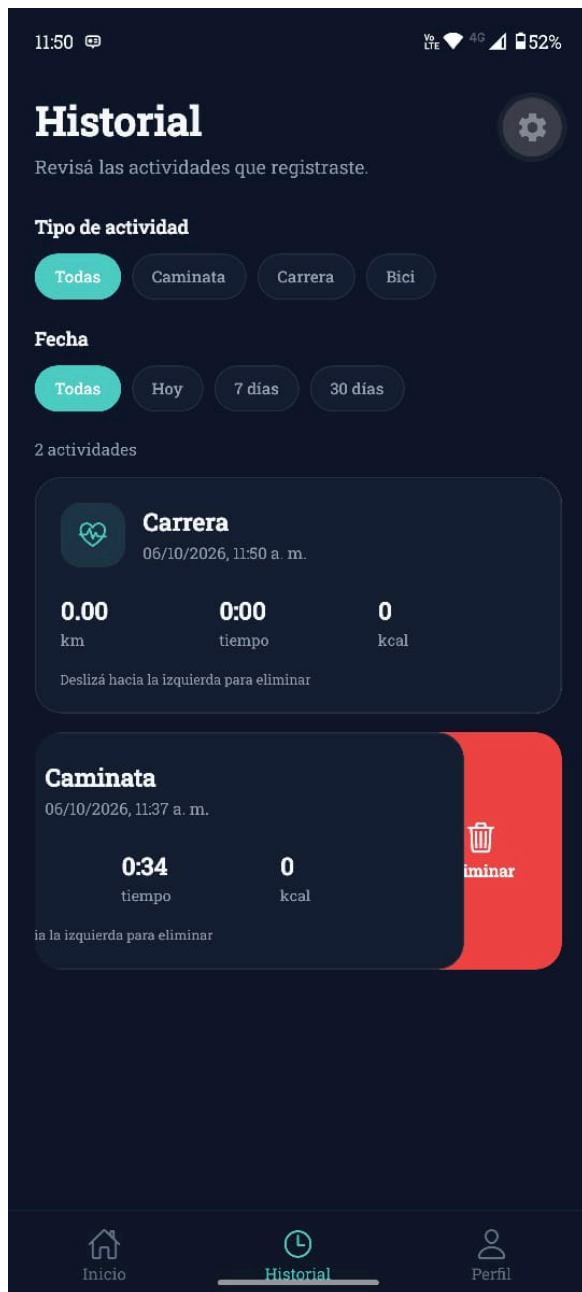
El usuario puede deslizar una tarjeta del historial hacia la izquierda.

Al realizar el movimiento aparece un fondo rojo con la opción:

Eliminar

Si el desplazamiento supera una distancia determinada, la actividad es eliminada.

El gesto fue desarrollado utilizando React Native Gesture Handler junto con React Native Reanimated.



11. Almacenamiento local

Las actividades son almacenadas localmente utilizando AsyncStorage.

Cada usuario posee una clave propia de almacenamiento.

Por ejemplo, conceptualmente:

`movemate_activities_ID_USUARIO`

Cuando no existe una sesión iniciada se utiliza:

`movemate_activities_guest`

Esto evita que las actividades de diferentes usuarios se mezclen.

El almacenamiento local también permite conservar información aunque no exista conexión a Internet.

12. Firebase y sincronización en la nube

MoveMate utiliza Firebase para permitir cuentas y sincronización.

Desde la sección Perfil el usuario puede:

- Crear una cuenta.
- Iniciar sesión.
- Cerrar sesión.

Cuando existe una cuenta autenticada, las actividades pueden almacenarse también en Cloud Firestore.

Cada documento se relaciona con el usuario mediante:

`userId`

Al consultar el historial, la aplicación puede recuperar actividades desde Firestore y almacenarlas también localmente.

De esta manera se combina:

`AsyncStorage`

+

Firebase

para mantener una copia local y otra en la nube.

13. Tema claro y oscuro

La aplicación permite utilizar tema claro y oscuro.

El sistema utiliza inicialmente el tema configurado en el dispositivo.

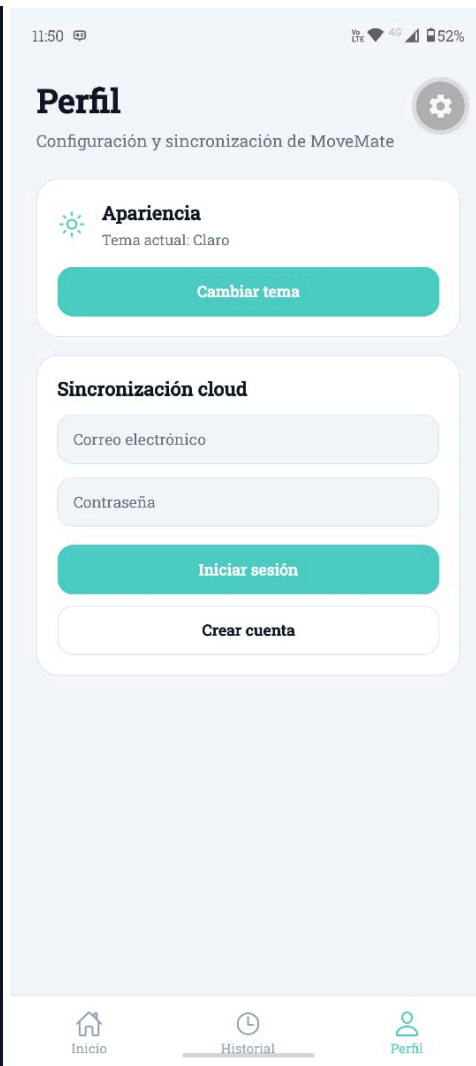
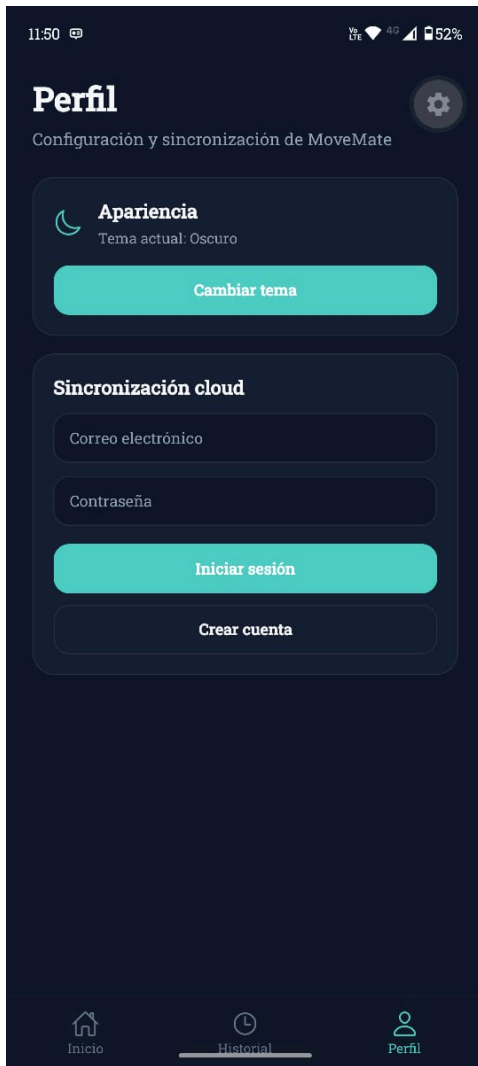
Desde la pantalla Perfil el usuario puede cambiar manualmente entre ambos modos.

La preferencia seleccionada se almacena mediante AsyncStorage.

Para administrar el tema globalmente se implementó un Context Provider.

Esto permite que el cambio afecte simultáneamente a:

- Inicio.
- Historial.
- Perfil.
- Navegación.
- Pantalla de actividad.



14. Generación de APK

Una vez desarrolladas las principales funcionalidades se configuró EAS Build.

Primero el proyecto fue vinculado con Expo mediante EAS CLI.

Posteriormente se configuró el archivo:

eas.json

con un perfil **preview** preparado para Android.

La compilación se inició mediante:

```
pnpm dlx eas-cli build -p android --profile preview
```

EAS realizó la compilación de manera remota.

También se generó automáticamente una clave de firma para Android.

El proceso terminó correctamente y produjo un archivo APK instalable.



15. Pruebas de funcionamiento

Durante el desarrollo se realizaron diferentes pruebas utilizando principalmente un dispositivo Android mediante Expo Go.

En la pantalla principal se verificó:

- Visualización del dashboard.
- Animaciones.
- Apertura del selector de actividad.
- Cambio de tema.

Durante una actividad se verificó:

- Solicitud de permisos.
- Obtención de ubicación.
- Temporizador.
- Registro de puntos.
- Cálculo de distancia.
- Cálculo de calorías.
- Visualización del mapa.
- Finalización de la actividad.

En el historial se verificó:

- Guardado de actividades.
- Persistencia local.
- Filtros.
- Swipe para eliminar.

En Firebase se verificó:

- Registro.
- Inicio de sesión.
- Cierre de sesión.
- Creación de documentos en Firestore.
- Asociación de actividades mediante `userId`.
- Recuperación de información.

También se comprobó el funcionamiento del tema claro y oscuro.

Finalmente se realizó una compilación Android mediante EAS Build.

16. Dificultades encontradas

Durante el desarrollo aparecieron diferentes problemas relacionados principalmente con navegación, mapas, Firebase y compilación.

Uno de los primeros problemas ocurrió al crear la ruta:

```
/activity/active
```

Expo Router no la reconocía inicialmente debido a que los tipos generados se encontraban desactualizados.

El problema se solucionó reiniciando Expo y el servidor de TypeScript.

Otro problema apareció con el sistema de temas.

Inicialmente cada pantalla utilizaba su propio estado, por lo que cambiar el tema desde Perfil no actualizaba el resto de la aplicación.

La solución fue crear un contexto global mediante:

```
AppThemeProvider
```

permitiendo que todas las pantallas utilizaran el mismo estado.

También aparecieron dificultades con Firebase Authentication.

Firebase mostraba una advertencia relacionada con la persistencia de sesión en React Native.

La versión utilizada no exponía correctamente algunas funciones de persistencia esperadas desde TypeScript, por lo que se mantuvo la autenticación mediante:

```
getAuth()
```

permitiendo utilizar normalmente registro, login y logout.

Otro problema importante estuvo relacionado con el mapa.

Inicialmente se utilizó:

```
react-native-maps
```

pero el mapa base no se visualizaba correctamente durante las pruebas.

También requería configuración adicional de Google Maps Platform.

Para evitar depender de ese servicio se reemplazó la implementación por:

- WebView.
- Leaflet.
- OpenStreetMap.

Con este cambio el mapa pudo mostrarse correctamente sin necesitar una API key de Google Maps.

Finalmente se configuró EAS Build.

Durante la primera compilación fue necesario:

- Vincular el proyecto con Expo.
- Configurar el identificador del proyecto.
- Corregir el slug.
- Crear un keystore para Android.
- Configurar el perfil `preview`.

Después de estos pasos, EAS pudo generar correctamente el APK.

17. Resultado final

Como resultado se obtuvo una aplicación móvil denominada **MoveMate**.

Actualmente la aplicación permite:

- Visualizar un dashboard.
- Mostrar estadísticas diarias.
- Utilizar animaciones.
- Iniciar caminatas.
- Iniciar carreras.
- Iniciar actividades en bicicleta.
- Solicitar permisos de ubicación.
- Obtener la ubicación mediante GPS.
- Registrar puntos del recorrido.
- Mostrar el recorrido en un mapa.
- Calcular distancia.
- Calcular calorías estimadas.
- Mostrar duración de la actividad.
- Guardar actividades.
- Consultar un historial.
- Filtrar por tipo.
- Filtrar por fecha.
- Eliminar actividades mediante swipe.
- Mantener información localmente.
- Crear cuentas.
- Iniciar y cerrar sesión.
- Guardar información en Firebase.
- Recuperar actividades desde Firestore.
- Separar las actividades entre diferentes usuarios.
- Cambiar entre tema claro y oscuro.
- Mantener la preferencia del tema.
- Generar una versión Android mediante EAS Build.

De esta manera se desarrolló una aplicación que combina seguimiento de actividad física, GPS, almacenamiento local, servicios cloud y diferentes componentes propios del desarrollo móvil.

18. Conclusión

El desarrollo de MoveMate permitió trabajar con diferentes conceptos relacionados con aplicaciones móviles utilizando React Native y Expo.

Uno de los puntos principales fue el uso de Expo Location para obtener datos de ubicación reales desde el dispositivo y utilizarlos para registrar una actividad.

También se trabajó con mapas mediante Leaflet y OpenStreetMap, permitiendo visualizar la posición y el recorrido realizado.

React Native Reanimated y Gesture Handler permitieron incorporar animaciones y gestos para mejorar la interacción con la aplicación.

Por otra parte, AsyncStorage permitió mantener información localmente, mientras que Firebase permitió incorporar autenticación y almacenamiento mediante Cloud Firestore.

El sistema de temas permitió aplicar un estado global compartido entre las diferentes pantallas.

Finalmente, EAS Build permitió trabajar con el proceso de compilación de una aplicación Android y generar un APK.

En conjunto, el proyecto permitió combinar interfaces móviles, GPS, mapas, animaciones, almacenamiento, servicios en la nube y compilación, obteniendo una aplicación orientada al registro de actividad física.